ADR-002: Search Features

October 3, 2025

Status: Proposed
Date: October 3, 2025

Context

We are integrating search capabilities to our decentralized model RSS music player so users can locate artists, songs, and feeds using a search term or metadata.

Searching for music feeds and Podcasting 2.0 Podcast feeds is possible with PodcastIndex API. Music tagged Podcasting 2.0 Podcast feeds are included. Podcast feeds along with Podcasting 2.0 Podcast feeds metadata can be searched as granular as episodes. Such an API does, however, provide limited calls, and complete dependency on it poses some risk.

A search index can be designed using the feeds already being fetched or PodcastIndex discovery can be balanced with the local search cache for a hybrid solution.

Key Drivers:

- User experience: Search feature must be efficient and effective
- Decentralization: Respect artist ownership and Podcasting 2.0 standards.
- Maintainability: Keep indexing and feed management simple.
- Reliability: Avoid outages or slow responses from external APIs.
- Cost: Keep infrastructure simple and inexpensive.

Key Non-Functional Requirements (NFRs)

- Performance: 95% of search queries should return results within 500 ms.
- Scalability: System can handle at least 1000 concurrent searches at a time.
- Availability: Service should remain operational 99.9% of the time, even if PodcastIndex is down.

- Compliance: Preserve original feed URLs, credits, and Podcasting 2.0 tags.
- Cost: Keep infrastructure and API costs reasonable/free.

Options

1. **Use PodcastIndex API directly.** Pros: Easy to implement, good coverage, no need for custom indexing. Cons: Introduces rate limits.
2. **Build a self-hosted search index.** Pros: Full control, no rate limits, not reliant on external services/hosting. Cons: More infrastructure, must handle indexing and ranking, may miss some feeds.
3. **Hybrid approach: PodcastIndex + local cache.** Pros: Uses caching to reduce API calls, improving speed and reliability. Cons: Adds more complexity, need to keep cache updated.

Decision

We will use the hybrid approach. We will use PodcastIndex for discovering feeds but store a local cache of common searches. This reduces API calls, handles rate limiting, and responds to frequent searches faster.

The local index will store basic metadata like feed titles, artist names, and track titles. We can use the RSS URL as the primary key for a database, and refresh the database regularly. PodcastIndex remains the main source for discovery.

Consequences

Positive:

- Fast, reliable search across many feeds.
- Less dependence on an external API.
- Handles rate limits and outages better.

Negative:

- More complex to maintain.
- Need to consistently update cache.
- Extra database and search infrastructure required.

Risks and Mitigations

- Risk: API rate limiting. Mitigation: Cache popular queries. Backoff/retry when limits are hit.
- Risk: Outdated cache. Mitigation: Refresh in the background. Include "last updated" timestamps.
- Risk: Extra infrastructure cost. Mitigation: Use lightweight tools like SQLite, and monitor usage.
- Risk: PodcastIndex changes terms or pricing. Mitigation: Local cache can be expanded to fully self-hosted search if needed.

References

- PodcastIndex API: https://podcastindex-org.github.io/docs-api/
- Podcasting 2.0 spec: https://podcasting2.org/docs

Contributions

I am contributing to the team by maintaining good communication with other members and attending all team meetings. Functionality-wise, I initially worked on the Track system. I added a Track component to display individual tracks and select the active track, created a UserFeed component to show a list of fetched tracks, and integrated UserFeed into the main page. Currently, I am working on implementing a navbar using Vue Bootstrap to maintain consistent styling across the page. I am collaborating closely with Spencer on the Bootstrap and styling, as he is responsible for the overall page styling.